

$$\begin{array}{lcl}
 y = x^2 & \rightarrow \text{program} \rightarrow (x) \rightarrow & \\
 \downarrow & & x \rightarrow \frac{dy}{dx} \\
 \frac{dy}{dx} = \boxed{2x} & \rightarrow \text{code} & \\
 \uparrow & & \\
 x \rightarrow 2 & \frac{dy}{dx} = 4 & \\
 x \rightarrow 3 & \frac{dy}{dx} = 6 & \\
 & & \frac{dy}{dx} = 2x \\
 y = x^2 & \xrightarrow{\quad} & x \rightarrow \frac{dz}{dx} \quad (2) \gg (1) \\
 z = \sin(y) & & \\
 \uparrow & & \boxed{\frac{dz}{dx}} = \frac{dz}{dy} \frac{dy}{dx} \\
 \frac{dz}{dy} = \cos y & & \\
 & \searrow & 2x \cos(y) = \boxed{2x \cos(x^2)}
 \end{array}$$

$$\left\{ \begin{array}{l} y = x^2 \\ z = \sin y \\ u = e^z \end{array} \right. \quad \frac{du}{dx} = \frac{du}{dz} \frac{dz}{dy} \frac{dy}{dx}$$

↓
code

[Nested function $\rightarrow$ complex $\rightarrow$ derivative $\rightarrow$ code difficult.]

Nested function $\rightarrow$ derivative

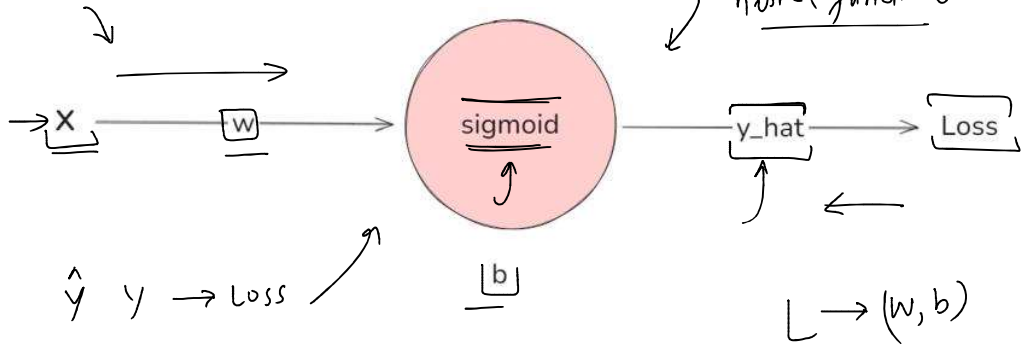
↓
[peephole]

$$\frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}$$

cgpa	placed
9.11	1
8.9	1
7	0
6.56	1
4.56	0

[Training process]

1. Forward pass - Compute the output of the network given an input.
2. Calculate loss - Calculate the loss function to quantify the error.
3. Backward pass - Compute gradients of the loss with respect to the parameters.
4. Update gradients - Adjust the parameters using an optimization algorithm (e.g., gradient descent).



Forward Pass Computation

1. Linear Transformation:

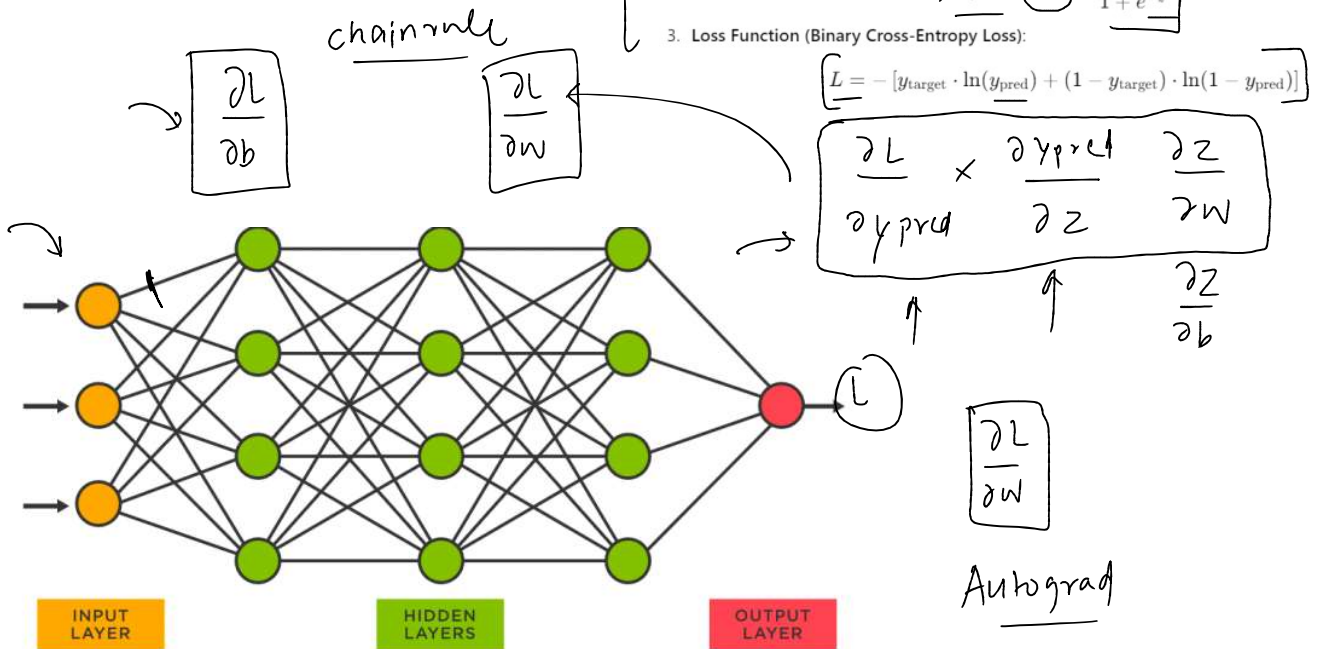
$$z = w \cdot x + b$$

2. Activation (Sigmoid Function):

$$y_{pred} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

3. Loss Function (Binary Cross-Entropy Loss):

$$L = -[y_{target} \cdot \ln(y_{pred}) + (1 - y_{target}) \cdot \ln(1 - y_{pred})]$$



NN $\rightarrow$ nested function
 $\hookrightarrow$ derivatives
 manually impossible

What is Autograd

28 November 2024 19:18

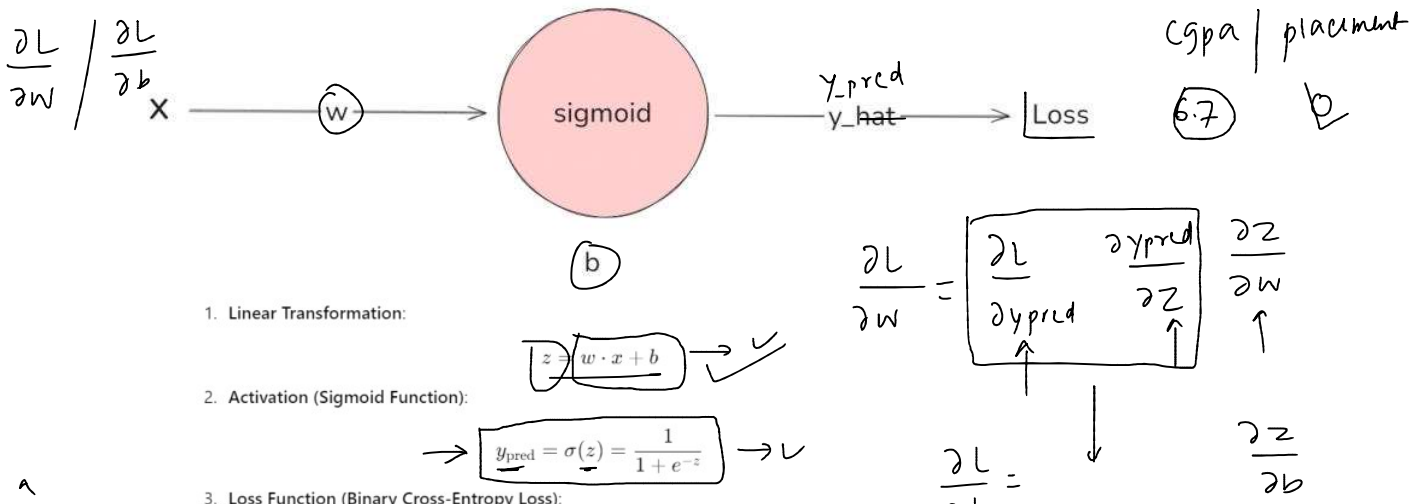
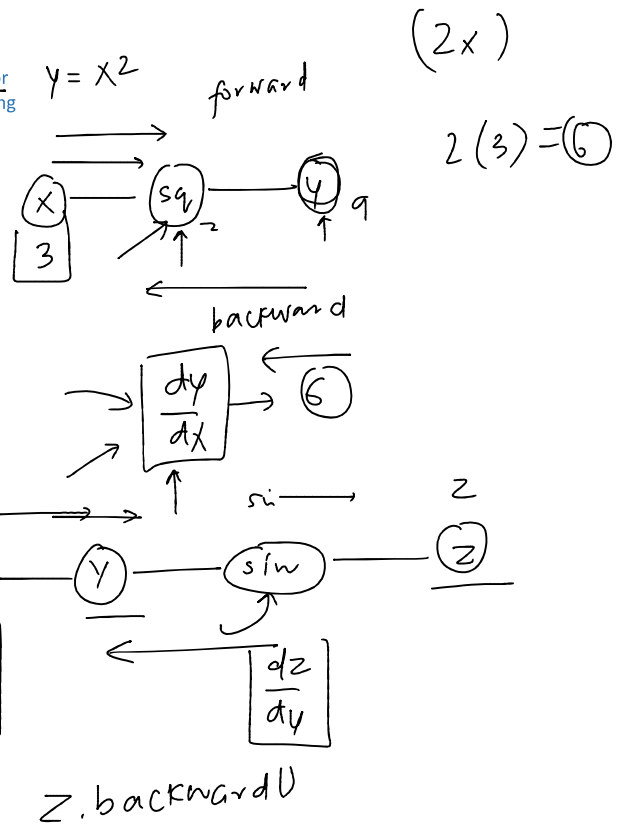
Autograd is a core component of PyTorch that provides automatic differentiation for tensor operations. It enables gradient computation, which is essential for training machine learning models using optimization algorithms like gradient descent.

Examples

- 1) $y = x^2$
- 2) $y = x^2$, $z = \sin(y) \rightarrow$
- 3) Neural network $\rightarrow$

$x = ?$ $\frac{dy}{dx}$

$x = 3$ $\frac{dz}{dx}$



$\hat{y}$ y_{pred}

$y_{pred} = \sigma(z) = \frac{1}{1 + e^{-z}} \rightarrow \checkmark$

3. Loss Function (Binary Cross-Entropy Loss):

$L = -[y_{target} \cdot \ln(y_{pred}) + (1 - y_{target}) \cdot \ln(1 - y_{pred})]$

$\frac{\partial L}{\partial y_{pred}} = \frac{(\hat{y} - y)}{\hat{y}(1 - \hat{y})}$

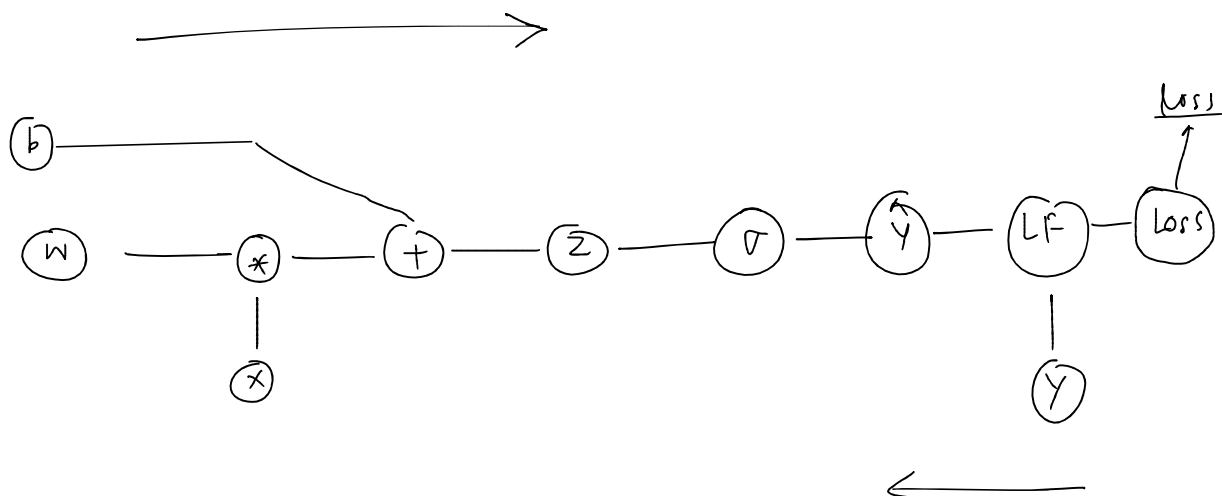
$\frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y})$

$\frac{\partial z}{\partial w} = x$

$\frac{\partial z}{\partial b} = 1$

$\frac{\partial L}{\partial w} = (\hat{y} - y) * x$

$\frac{\partial L}{\partial b} = (\hat{y} - y) * 1$



$x = \begin{bmatrix} x_1 & x_2 & x_3 \\ 1 & 2 & 3 \end{bmatrix}$

$y = x^2 \cdot \text{mean}()$

$y = f(x_1, x_2, x_3)$

multivar fund.

$y = \frac{x_1^2 + x_2^2 + x_3^2}{3}$

$\frac{\partial y}{\partial x_1} = \frac{2x_1}{3}$

$\frac{\partial y}{\partial x_2} = \frac{2x_2}{3}$

$\frac{\partial y}{\partial x_3} = \frac{2x_3}{3}$

$$0.15' \quad \downarrow \quad \left(\frac{2}{3} \right) = 0.75$$

$$\frac{4}{3} = 1.33 \quad 2$$